

# Artec 3D

## Senior Front-End

Question-by-question knowledge base · React · Rendering · SSR · Web performance

### CONTENTS

1. React Hooks
2. The JavaScript event loop
3. React vs Next.js for SEO
4. The main advantage of SSR
5. Rendering a list of ~1M items
6. Preventing unnecessary re-renders
7. What triggers a re-render
8. How React decides what to update (VDOM & reconciliation)
9. Migrating an older React app to React 18
10. Code-quality tooling
11. What REST stands for
12. Web Vitals / Core Web Vitals

## What are React Hooks?

### React fundamentals

Functions that let you use React features — state, lifecycle, context — inside **function components**, without writing a class. They were introduced in React 16.8.

Core hooks worth naming:

- `useState` — local component state.
- `useEffect` — side effects (subscriptions, fetching, manual DOM work), run after render.
- `useContext` — read context without a consumer wrapper.
- `useRef` — a mutable value that persists across renders without triggering one; also DOM refs.
- `useMemo` / `useCallback` — memoize a computed value / a function identity.
- `useReducer` — state via a reducer, for more complex transitions.

### RULES OF HOOKS

Only call them at the **top level** (never inside conditions, loops, or nested functions) and only from React function components or custom hooks. This is what lets React match each hook call to its stored state by call order.

Custom hooks (a function named `useX` that calls other hooks) are how you extract and reuse stateful logic.

## How does the JavaScript event loop work?

### JS runtime

JavaScript runs on a **single thread** with one call stack. The event loop is the mechanism that lets it handle async work without blocking that thread.

The pieces:

- **Call stack** — synchronous code executes here, one frame at a time.
- **Web/host APIs** — timers, fetch, DOM events run outside the engine (browser/Node), and hand a callback back when done.
- **Task queue (macrotasks)** — `setTimeout`, I/O, UI events.
- **Microtask queue** — resolved Promise callbacks ( `.then`, `await` continuations), `queueMicrotask`, `MutationObserver`.

The loop: when the call stack is empty, it drains **all** microtasks first, then takes **one** macrotask, then drains microtasks again — and repeats. So microtasks always run before the next timer callback.

### THE CLASSIC PROOF

`setTimeout(...,0)` logs *after* a `Promise.resolve().then(...)`, even though both look "immediate" — the promise is a microtask, the timeout is a macrotask.

## Does plain React or Next.js have an advantage for SEO?

SSR / Next.js

**Next.js has the advantage.** A plain React app (a client-side SPA, e.g. Vite/CRA) ships a near-empty HTML shell and builds the page in the browser with JavaScript. Crawlers that don't fully execute JS — or execute it late — may see little or no content.

Next.js renders the HTML on the **server** (SSR) or at **build time** (SSG), so the crawler receives fully-formed markup — real content, meta tags, titles — on the first response.

Why it helps ranking:

- Content is present without depending on client JS execution.
- Per-page `<title>` / meta / Open Graph via the metadata API.
- Faster first paint improves Core Web Vitals, which feed ranking.

*Nuance to mention:* Google can render JS, so SPAs aren't invisible — but it's slower, less reliable, and worse for non-Google crawlers and social preview bots. Server-rendered HTML is simply the safer default for content that must be indexed.

---

## What is the main advantage of SSR?

SSR / Next.js

The page arrives as **meaningful HTML on the first response**, instead of a blank shell the browser has to fill in. That gives you two big wins:

- **Faster perceived load / first contentful paint** — the user sees real content before the JS bundle downloads and executes.
- **Indexability & link previews** — crawlers and social bots get complete content immediately.

Then **hydration** attaches React's event handlers to that server-rendered markup to make it interactive.

### TRADE-OFF TO ACKNOWLEDGE

SSR adds server cost and per-request latency (TTFB), and the app must be able to run in a non-browser environment. It's best where content and SEO matter; a highly interactive internal dashboard behind a login gains far less from it.

## How would you render a scrollable list of ~1 million items?

### Performance

**Virtualization (windowing).** You never put a million nodes in the DOM. You render only the handful of rows visible in the viewport (plus a small overscan buffer), and swap their content as the user scrolls.

How it works:

- A tall spacer element gives the scrollbar the correct total height.
- On scroll, compute which index range is visible from scroll offset ÷ row height.
- Render only those rows, absolutely positioned at their offset.

In practice you reach for a library rather than hand-rolling it: **TanStack Virtual** (headless, and on-stack given the Artec role), or `react-window` / `react-virtualized`.

Complementary techniques to mention:

- **Pagination / infinite scroll** — don't even fetch a million rows; load in chunks (pairs well with TanStack Query).
- Stable `key`s and memoized row components so scrolling doesn't thrash.
- Variable row heights need a measuring/dynamic virtualizer.

### ONE-LINER

The DOM is the bottleneck, not the data — keep the number of mounted nodes constant regardless of list length.

## How can unnecessary re-renders be prevented?

### React rendering

Start by knowing the default: when a component renders, React re-renders its whole subtree. Most "unnecessary" renders come from new prop/reference identities or state living too high up.

Tools, roughly in order of reach:

- **React.memo** — skips re-rendering a component when its props are shallow-equal to last time.
- **useMemo** — caches an expensive computed value between renders.
- **useCallback** — keeps a function's identity stable so memoized children don't see a "new" prop.
- **Stable keys** — correct keys avoid remounting list items.
- **Lift state down / colocate it** — keep state as local as possible so fewer components re-render.
- **Split context / select slices** — a single big context re-renders every consumer on any change; split it, or use a selector-based store (Redux Toolkit, Zustand).
- **Pass children as props** — content passed via **children** isn't re-created when the parent re-renders.

### SAY THIS

Don't sprinkle **memo** / **useMemo** everywhere by reflex — measure with the React DevTools Profiler, then memoize the hot paths. Premature memoization adds its own cost and clutter. (Also worth noting: the React Compiler in React 19 automates much of this.)

## What triggers a component / UI re-render?

### React rendering

Four things:

- **State change** — calling a `useState` / `useReducer` setter (with a value that differs by `Object.is` ).
- **New props** — the parent re-rendered and passed down props.
- **Parent re-render** — a parent rendering re-renders its children by default, even if their props didn't change (unless memoized).
- **Context change** — a consumed context value updated.

Notably *not* triggers: mutating a `ref` , or mutating an object/array in place without calling a setter. React compares the **reference**, so an in-place mutation that keeps the same reference won't re-render — which is exactly why immutable updates matter.

---



## How does React decide what to update after a state change?

React rendering · VDOM

Via the **Virtual DOM and reconciliation**.

- Rendering produces a new **virtual DOM** tree — a lightweight JS description of the UI.
- React **diffs** that new tree against the previous one (reconciliation).
- It computes the minimal set of real DOM operations and commits only those. Untouched nodes are left alone.

The diffing relies on two heuristics: elements of a **different type** are replaced wholesale (subtree rebuilt), and **keys** identify which children are the same across renders so items can be reordered instead of destroyed and recreated. Bad keys (e.g. array index on a reordering list) are a common source of subtle bugs.

### ONE-LINER

The virtual DOM makes "figure out what changed" cheap; direct DOM manipulation is the expensive part, so React minimizes it.

*If pushed on modern internals:* since React 16 the reconciler ("Fiber") can split work into units and prioritize/interrupt it — the basis for concurrent features.

## How would you migrate an older React app to React 18?

### Migration

Frame it as **small, safe, shippable steps** — which is exactly the discipline this role is hiring for.

- **Bump the deps** — `react` and `react-dom` to 18, plus any libraries that need a compatible major (routing, state, testing).
- **Switch to the new root API** — replace `ReactDOM.render(...)` with `createRoot(container).render(...)`. This is what opts you into React 18 behaviour and concurrent features.
- **Expect automatic batching** — 18 batches state updates inside promises, timeouts and native handlers too, not just React events. Usually a win; occasionally surfaces code that relied on the old timing.
- **Test under `StrictMode`** — in dev it intentionally double-invokes effects/renders to flush out unsafe side effects. Fix effects that aren't idempotent or lack cleanup.
- **Migrate incrementally & lean on the type checker + tests** — upgrade, run the suite, fix warnings, ship; don't rewrite features at the same time.
- **Adopt new APIs later, deliberately** — `useTransition`, `useDeferredValue`, Suspense for data, `useId` — once you're stable on 18.

### SAY THIS

"Get to 18 running unchanged first (root API + green tests), then adopt concurrent features as separate, reviewable changes." Note this maps directly onto Artec's CRM, which is on React 18 already and migrating routing to TanStack Router.

## Which tools/checks help guarantee code quality?

### Tooling / CI

Layer them so problems are caught progressively earlier:

- **TypeScript** — static type checking ( `tsc --noEmit` ) catches whole classes of bugs before runtime.
- **ESLint** — catches bugs and enforces conventions; the `react-hooks` plugin flags Rules-of-Hooks and effect-dependency mistakes.
- **Prettier** — formatting, so reviews are about substance, not style.
- **Tests** — unit/component (Vitest/Jest + React Testing Library), plus E2E (Playwright/Cypress) for critical flows.
- **Visual regression** — Storybook + Chromatic to catch unintended UI changes (on-stack for Artec UI).
- **Pre-commit hooks** — Husky + lint-staged run lint/format on changed files before a commit lands.
- **CI** — run type-check, lint, tests and build on every PR so nothing merges red; require review.

### FRAMING

Local hooks give fast feedback; CI is the enforced gate. Same checks, two places.

## What does REST stand for?

### APIs

**RE**presentational **S**tate **T**ransfer — an architectural style for networked APIs, typically over HTTP.

Worth adding a sentence of substance so it's not just the acronym:

- **Resources** identified by URLs ( `/users/42` ), acted on with HTTP methods: `GET` , `POST` , `PUT` / `PATCH` , `DELETE` .
- **Stateless** — each request carries everything needed; the server keeps no client session between calls.
- Meaningful **status codes** (2xx/4xx/5xx) and representations, usually JSON.

*Optional contrast:* versus GraphQL (one endpoint, client-specified shape) — REST is resource-and-verb oriented with multiple endpoints.

---

## What are Web Vitals / Core Web Vitals?

### Web performance

Google's standardized metrics for real-world user experience. **Core** Web Vitals are the three headline ones, each measuring a different dimension — loading, interactivity, visual stability:

- **LCP — Largest Contentful Paint** (loading): time until the largest visible element renders. Good  $\leq 2.5s$ .
- **INP — Interaction to Next Paint** (interactivity): responsiveness across all interactions; it replaced FID in 2024. Good  $\leq 200ms$ .
- **CLS — Cumulative Layout Shift** (visual stability): how much content unexpectedly jumps. Good  $\leq 0.1$ .

Supporting (non-core) vitals you can name: **FCP** (First Contentful Paint) and **TTFB** (Time To First Byte).

### WHY IT MATTERS HERE

They're measurable, they feed search ranking, and SSR/virtualization/render-optimization (the earlier questions) are precisely the levers that move them — LCP via server rendering, INP via less main-thread work, CLS via reserving space for images/async content.